

```

package projem1;

import javax.swing.*.*;
import java.awt.*.*;
import java.awt.event.*.*;

class EtkilesimliAracKutusu extends JFrame
    implements ActionListener, ItemListener, MouseListener,
        MouseMotionListener, KeyListener, WindowListener {

    // Bileşenler
    private JMenuBar menuBar;
    private JMenu dosyaMenu, yardimMenu;
    private JMenuItem cikisItem, hakkimdaItem;

    private JTextField txtAd;
    private JRadioButton rbSiyah, rbKirmizi, rbMavi;
    private ButtonGroup renkGrubu;
    private JCheckBox chkBilgiGoster;
    private JButton btnFareTest, btnSagTikla;
    private JLabel lblFareKoordinat, lblDurum;
    private JTextField txtTus;
    private JButton btnTemizle;

    JButton keyfiButon, keyfiButon2;

    Gorev g1=null;

    public EtkilesimliAracKutusu() {
        setTitle("Etkileşimli Araç Kutusu - Listener Demo");
        setSize(600, 450);
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(null);

        // ===== MENU BAR =====
        menuBar = new JMenuBar();

        dosyaMenu = new JMenu("Dosya");
        cikisItem = new JMenuItem("Çıkış");
        cikisItem.addActionListener(this);
        dosyaMenu.add(cikisItem);

        yardimMenu = new JMenu("Yardım");
        hakkimdaItem = new JMenuItem("Hakkında");
        hakkimdaItem.addActionListener(this);
        yardimMenu.add(hakkimdaItem);

        menuBar.add(dosyaMenu);
        menuBar.add(yardimMenu);
        setJMenuBar(menuBar);

        // ===== BİLEŞENLER =====

        // Ad alanı
        JLabel lblAd = new JLabel("Adınız:");
        lblAd.setBounds(30, 50, 80, 25);
        add(lblAd);

        txtAd = new JTextField();

```

```
txtAd.setBounds(120, 50, 200, 25);
add(txtAd);

// RadioButton'lar (ItemListener ile)
JLabel lblRenk = new JLabel("Renk Seç:");
lblRenk.setBounds(30, 90, 80, 25);
add(lblRenk);

rbSiyah = new JRadioButton("Siyah");
rbKirmizi = new JRadioButton("Kırmızı");
rbMavi = new JRadioButton("Mavi");
rbSiyah.setBounds(120, 90, 70, 25);
rbKirmizi.setBounds(200, 90, 80, 25);
rbMavi.setBounds(290, 90, 70, 25);

// ItemListener ekle
rbSiyah.addItemListener(this);
rbKirmizi.addItemListener(this);
rbMavi.addItemListener(this);

renkGrubu = new ButtonGroup();
renkGrubu.add(rbSiyah);
renkGrubu.add(rbKirmizi);
renkGrubu.add(rbMavi);

add(rbSiyah);
add(rbKirmizi);
add(rbMavi);

// CheckBox (ItemListener ile)
chkBilgiGoster = new JCheckBox("Bilgi Göster");
chkBilgiGoster.setBounds(30, 130, 150, 25);
chkBilgiGoster.addItemListener(this);
add(chkBilgiGoster);

// Butonlar (MouseListener için)
btnFareTest = new JButton("Fareyle Üzerime Gel");
btnFareTest.setBounds(30, 170, 180, 35);
btnFareTest.addMouseListener(this);
add(btnFareTest);

btnSagTikla = new JButton("Sağ Tıkla");
btnSagTikla.setBounds(230, 170, 120, 35);
btnSagTikla.addMouseListener(this);
add(btnSagTikla);

// Fare koordinatları (MouseMotionListener için)
lblFareKoordinat = new JLabel("Fare Koordinatları: (0, 0)");
lblFareKoordinat.setBounds(30, 220, 250, 25);
lblFareKoordinat.setBorder(BorderFactory.createLineBorder(Color.GRAY));
add(lblFareKoordinat);

// MouseMotionListener'ı JFrame'e ekle
addMouseMotionListener(this);

// KeyListener için text alanı
JLabel lblTus = new JLabel("Tuşa Bas:");
lblTus.setBounds(30, 260, 80, 25);
add(lblTus);

txtTus = new JTextField();
txtTus.setBounds(120, 260, 150, 25);
txtTus.addKeyListener(this);
add(txtTus);
```

```

// Temizle butonu
btnTemizle = new JButton("TEMİZLE");
btnTemizle.setBounds(30, 310, 120, 35);
btnTemizle.addActionListener(this);
add(btnTemizle);

// Durum label'ı (EN SON oluştur)
lblDurum = new JLabel("Durum: Bekleniyor...");
lblDurum.setBounds(30, 360, 400, 25);
lblDurum.setForeground(Color.BLUE);
add(lblDurum);

// WindowListener ekle
addWindowListener(this);

keyfiButon=new JButton("Keyfi buton");
Rectangle r=btnTemizle.getBounds();
keyfiButon.setBounds(r.x+200,r.y,r.width,r.height);
keyfiButon.addActionListener(this);
add(keyfiButon);

keyfiButon2=new JButton("Keyfi2 buton");
Rectangle r2=keyfiButon.getBounds();
keyfiButon2.setBounds(r2.x+200,r2.y,r2.width,r2.height);
keyfiButon2.addActionListener(this);
add(keyfiButon2);

// *** DÜZELTME: setSelected işlemlerini EN SON yap ***
rbSiyah.setSelected(true);

setVisible(true);
}

// ===== ACTION LISTENER =====
@Override
public void actionPerformed(ActionEvent e) {

    if(e.getSource().equals(keyfiButon)) {
        g1=new Gorev(2000);
        Thread t1=new Thread(g1);
        t1.start();
    }
    else if(e.getSource().equals(keyfiButon2)) {
        if(g1!=null)
            g1.calis=!g1.calis;
    }
    else if (e.getSource() == cikisItem) {
        //dispose();
        System.exit(0);
    } else if (e.getSource() == hakkimdaItem) {
        JOptionPane.showMessageDialog(this,
            "Etkileşimli Araç Kutusu\\n\\n" +
            "Kullanılan Listener'lar:\\n" +
            "• ActionListener (menüler, butonlar)\\n" +
            "• ItemListener (radio, checkbox)\\n" +
            "• MouseListener (butonlarda)\\n" +
            "• MouseMotionListener (fare takibi)\\n" +
            "• KeyListener (text alanı)\\n" +
            "• WindowListener (pencere olayları)");
    }
}

```

```

    } else if (e.getSource() == btnTemizle) {
        txtAd.setText("");
        txtTus.setText("");
        lblFareKoordinat.setText("Fare Koordinatları: (0, 0)");
        renkGrubu.clearSelection();
        rbSiyah.setSelected(true);
        chkBilgiGoster.setSelected(false);
        btnFareTest.setBackground(null);
        btnFareTest.setForeground(null);
        lblDurum.setText("Durum: Temizlendi");
    }
}

// ===== ITEM LISTENER (radio ve checkbox için) =====
@Override
public void itemStateChanged(ItemEvent e) {
    // lblDurum null kontrolü (güvenlik için)
    if (lblDurum == null) return;

    if (e.getSource() == rbSiyah && e.getStateChange() ==
ItemEvent.SELECTED) {
        lblDurum.setText("Durum: Siyah renk seçildi");
        btnFareTest.setBackground(Color.BLACK);
        btnFareTest.setForeground(Color.WHITE);
    } else if (e.getSource() == rbKirmizi && e.getStateChange() ==
ItemEvent.SELECTED) {
        lblDurum.setText("Durum: Kırmızı renk seçildi");
        btnFareTest.setBackground(Color.RED);
        btnFareTest.setForeground(Color.WHITE);
    } else if (e.getSource() == rbMavi && e.getStateChange() ==
ItemEvent.SELECTED) {
        lblDurum.setText("Durum: Mavi renk seçildi");
        btnFareTest.setBackground(Color.BLUE);
        btnFareTest.setForeground(Color.WHITE);
    } else if (e.getSource() == chkBilgiGoster) {
        if (chkBilgiGoster.isSelected()) {
            lblDurum.setText("Durum: Bilgi göster modu AÇIK");
            JOptionPane.showMessageDialog(this, "Bilgi gösterimi aktif!");
        } else {
            lblDurum.setText("Durum: Bilgi göster modu KAPALI");
        }
    }
}

// ===== MOUSE LISTENER =====
@Override
public void mouseClicked(MouseEvent e) {
    if (lblDurum == null) return;

    if (e.getSource() == btnSagTikla && e.getButton() == MouseEvent.BUTTON3)
{
        lblDurum.setText("Durum: Sağ tıklandı!");
        JOptionPane.showMessageDialog(this, "Butona sağ tıkladınız!");
    } else if (e.getSource() == btnFareTest && e.getButton() ==
MouseEvent.BUTTON1) {
        lblDurum.setText("Durum: Sol tık - butona tıklandı");
    }
}

@Override
public void mouseEntered(MouseEvent e) {
    if (lblDurum == null) return;

    if (e.getSource() == btnFareTest) {

```

```

        lblDurum.setText("Durum: Fare butonun ÜZERİNDE");
        btnFareTest.setToolTipText("Fareniz üzerimde!");
    }
}

@Override
public void mouseExited(MouseEvent e) {
    if (lblDurum == null) return;

    if (e.getSource() == btnFareTest) {
        lblDurum.setText("Durum: Fare butondan ÇIKTI");
    }
}

@Override
public void mousePressed(MouseEvent e) {}

@Override
public void mouseReleased(MouseEvent e) {}

// ===== MOUSE MOTION LISTENER =====
@Override
public void mouseMoved(MouseEvent e) {
    if (lblFareKoordinat != null) {
        lblFareKoordinat.setText(String.format("Fare Koordinatları: (%d, %d)", e.getX(), e.getY()));
    }
}

@Override
public void mouseDragged(MouseEvent e) {
    if (lblFareKoordinat != null && lblDurum != null) {
        lblFareKoordinat.setText(String.format("SÜRÜKLENİYOR: (%d, %d)", e.getX(), e.getY()));
        lblDurum.setText("Durum: Fare sürükleniyor");
    }
}

// ===== KEY LISTENER =====
@Override
public void keyPressed(KeyEvent e) {
    if (lblDurum == null) return;

    lblDurum.setText("Durum: Tuşa basıldı - " +
        KeyEvent.getKeyText(e.getKeyCode()));

    if (e.getKeyCode() == KeyEvent.VK_ENTER) {
        btnFareTest.setBackground(Color.GREEN);
        lblDurum.setText("Durum: ENTER tuşuna basıldı! Buton yeşil oldu.");
    }

    if (e.getKeyCode() == KeyEvent.VK_ESCAPE) {
        txtTus.setText("");
        lblDurum.setText("Durum: ESC tuşu - temizlendi");
    }
}

@Override
public void keyReleased(KeyEvent e) {
    if (lblDurum != null) {
        lblDurum.setText("Durum: Tuş bırakıldı - " +
            KeyEvent.getKeyText(e.getKeyCode()));
    }
}

```

```

@Override
public void keyTyped(KeyEvent e) {
    if (txtTus != null) {
        txtTus.setText("'" + e.getKeyChar() + "' tuşuna basıldı");
    }
}

// ===== WINDOW LISTENER =====
@Override
public void windowOpened(WindowEvent e) {
    if (lblDurum != null) {
        lblDurum.setText("Durum: Pencere AÇILDI - Hoş geldiniz!");
    }
    System.out.println("windowOpened çalıştı");
}

@Override
public void windowClosing(WindowEvent e) {
    int cevap = JOptionPane.showConfirmDialog(this,
        "Çıkmak istediğinize emin misiniz?",
        "Çıkış Onayı",
        JOptionPane.YES_NO_OPTION);
    if (cevap == JOptionPane.YES_OPTION) {
        if (lblDurum != null) {
            lblDurum.setText("Durum: Pencere kapatılıyor...");
        }
        dispose();
        System.exit(0);
    } else {
        if (lblDurum != null) {
            lblDurum.setText("Durum: Çıkış iptal edildi");
        }
    }
}

@Override
public void windowClosed(WindowEvent e) {
    System.out.println("windowClosed çalıştı");
}

@Override
public void windowIconified(WindowEvent e) {
    if (lblDurum != null) {
        System.out.println("Durum: Pencere KÜÇÜLTÜLDÜ");
    }
}

@Override
public void windowDeiconified(WindowEvent e) {
    if (lblDurum != null) {
        System.out.println("Durum: Pencere GERİ YÜKLENDİ");
    }
}

@Override
public void windowActivated(WindowEvent e) {
    if (lblDurum != null) {
        System.out.println("Durum: Pencere AKTİF");
    }
}

@Override
public void windowDeactivated(WindowEvent e) {

```

```

        if (lblDurum != null) {
            System.out.println("Durum: Pencere PASİF");
        }
    }
}

class Gorev implements Runnable{
    long beklemeSuresi=1000;
    volatile boolean calis=true; //flag

    Gorev(int bS){
        beklemeSuresi=bS;
    }

    @Override
    public void run() {
        // TODO Auto-generated method stub
        for(int i=0;i<1000000000L;) {

            if(calis) {
                System.out.println(i);
                try {
                    Thread.sleep(beklemeSuresi);
                } catch (InterruptedException e) {
                    // TODO Auto-generated catch block
                    e.printStackTrace();
                }
                i++;
            }

        }
    }
}

public class projem1{
    public static void main(String[] args) {
        new EtkilesimliAracKutusu();
    }
}

```